CHAPTER 6

ALU Design

Full adder · ripple vs carry-lookahead · adder/subtractor · overflow flag · Booth's multiplication · division

QUICK TRICK

Full adder: $\mathbf{S = A \oplus B \oplus C_{in}}$, $\mathbf{C_{out} = AB + C_{in}(A \oplus B)}$. Carry lookahead: $\mathbf{G = AB}$, $\mathbf{P = A \oplus B}$, $\mathbf{C_{i+1} = G_i + P_iC_i}$. Subtract = add B' with $C_{in} = 1$. Booth: 10 → subtract M, 01 → add M, 00/11 → shift only.

6.1 What does an ALU do?

● LEVEL 1 · BEGINNER

The ALU performs arithmetic (add, subtract, multiply) and logic (AND, OR, XOR, NOT, shift). Its building block is the **1-bit adder**. Flags it sets: **Z** (zero), **N** (negative), **C** (carry), **V** (overflow).

6.2 Adders

● LEVEL 2 · EXAM

Circuit	Sum	Carry out
Half adder	$A \oplus B$	$A \cdot B$
Full adder	$A \oplus B \oplus C_{in}$	$AB + C_{in}(A \oplus B)$

- **Ripple-carry adder:** n full adders in a chain. The carry must travel through every stage, so delay grows **linearly with n**.
- **Carry-lookahead adder (CLA):** compute all carries in parallel from **generate** $\mathbf{G_i = A_iB_i}$ and **propagate** $\mathbf{P_i = A_i \oplus B_i}$:

```
C1 = G0 + P0 C0
C2 = G1 + P1 G0 + P1 P0 C0
C3 = G2 + P2 G1 + P2 P1 G0 + P2 P1 P0 C0          Si = Pi XOR Ci
Delay becomes (nearly) independent of n for a block; wider adders use groups of lookahead blocks.
```

Example. A = 1011 (11), B = 0110 (6), $C_0 = 0$. Bits LSB first: G = 0,1,0,0; P = 1,0,1,1. C1 = 0, C2 = 1, C3 = 1, C4 = 1. S = P ⊕ C = 1,0,0,0. Result = C4 S3 S2 S1 S0 = **10001 = 17** ✓.

6.3 Adder/subtractor and overflow

● LEVEL 2 · EXAM

```
B_i ---XOR---> to adder        M = 0 : A + B            M = 1 : A - B = A + B' + 1
M   ----^----> (also Cin)      n XOR gates, Cin = M
Overflow V = C_into_MSB XOR C_out_of_MSB
```

6.4 Multiplication

● LEVEL 3 · GATE

- **Shift-and-add (unsigned):** for each multiplier bit that is 1, add the shifted multiplicand. n-bit × n-bit gives a **2n-bit** product; n steps.
- **Array multiplier (n × n):** n^2 AND gates, $n(n-2)$ full adders, n half adders. (8 × 8: 64 AND, 48 FA, 8 HA.)

Booth's algorithm (signed multiplication)

Look at pairs (Q_0, Q_{-1}) of the multiplier (Q_{-1} starts at 0). **10:** A = A − M. **01:** A = A + M. **00 or 11:** do nothing. Then **arithmetic shift right** A, Q, Q_{-1}. Repeat n times. Result is in A:Q.

Example. M = 0011 (+3), Q = 1110 (−2), n = 4:

AMJESH (https://singhamjesh.github.io/)

```
Step   A     Q     Q-1   Action
init   0000  1110  0
1      0000  0111  0     pair 00 -> shift
2      1110  1011  1     pair 10 -> A = A - M = 1101, shift
3      1111  0101  1     pair 11 -> shift
4      1111  1010  1     pair 11 -> shift
Product = A:Q = 1111 1010 = -6
```

QUICK TRICK

Booth's trick: a run of 1s is worth $(+2^{k+1} - 2^{j})$, so one subtraction at the run's start and one addition after its end. A multiplier like 00111100 needs only **2** add/subtract operations.

6.5 Division

● LEVEL 2 · EXAM

Restoring division: n steps. Shift the remainder:quotient left, subtract the divisor; if the result is negative restore it and set quotient bit 0, else keep it and set bit 1. **Non-restoring** avoids the restore step (it adds instead in the next step). Both take n iterations for n-bit operands.

6.6 Common mistakes

COMMON MISTAKE

- Saying a CLA removes the carry delay entirely (it reduces it; very wide adders use groups).
- Forgetting $C_{in} = 1$ when using the adder for subtraction.
- Using a 1-bit product width: n × n multiplication needs **2n** bits.
- Doing a logical shift instead of an **arithmetic** shift in Booth.

GATE QUICK CHECK — Chapter 6

```
FA: S=A^B^Cin  Cout=AB+Cin(A^B)      CLA: G=AB, P=A^B, C(i+1)=Gi+Pi*Ci, Si=Pi^Ci
A-B = A + B' + 1 (XOR gates + Cin=1)     V = C_in_MSB XOR C_out_MSB
Booth pair (Q0,Q-1): 10 sub | 01 add | 00,11 shift      product = 2n bits, n steps
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Sum and carry of a full adder for A = 1, B = 1, $C_{in} = 1$.

Q2. Why is a ripple-carry adder slow?

Q3. How many bits does the product of two 8-bit numbers need?

Q4. How does an adder subtract?

● LEVEL 2 · EXAM

Q5. 4-bit addition 0110 + 0101. Overflow flag?

Q6. Booth multiplication of (+3) × (−2) in 4 bits. Result?

Q7. How many add/subtract operations does Booth perform for multiplier 00111100?

Q8. CLA: A = 1011, B = 0110, $C_0 = 0$. Give P, G and the sum.

● LEVEL 3 · GATE

Q9. Number of gates in an 8 × 8 array multiplier (AND, full adders, half adders).

Q10. Restoring division of 8-bit numbers: how many iterations?

Q11. How many XOR gates does an n-bit adder/subtractor need (excluding overflow logic)?

Answers

Q	Solution
1	S = 1, C_{out} = **1**.
2	The carry passes through every stage, so delay grows with the number of bits.
3	**16**.
4	A + B' + 1: XOR B with 1 and set C_{in} = 1.
5	Sum 1011. Carry into MSB 1, carry out 0, so **V = 1**.
6	**1111 1010 = −6** (trace in 6.4).
7	**2** (one subtract at the start of the run of 1s, one add after its end).
8	P = 1101, G = 0010 (reading bit 3 to bit 0 gives P: 1,1,0,1 and G: 0,0,1,0). Sum = **10001** (17).
9	**64 AND, 48 full adders, 8 half adders**.
10	**8**.
11	**n** (one per bit of B).

Next: Chapter 7 — Control Unit: Hardwired and Microprogrammed.

AMJESH
singhamjesh.github.io